Guitare électrique : Etude de l'influence du type de bobinage dans le micro

Christophe Morel

Lycée Ferdinand Buisson, dit ''la Nat'', CPGE PT

Annexes

Code Python

Table des matières

TABLE DES MATIERES 2

I. MODELISATION.PY 3

I.I. FONCTION UTILES 3

I.II. ALGORITHME DE LEVENBERG-MARQARDT 5

I.III. FONCTIONS POUR LA MODELISATION ACOUSTIQUE 10

I.IV. FONCTION POUR LA MODELISATION MAGNETIQUE 12

II. TRAITEMENT DES DONNEES EXPERIMENTALE 13

III. REPONSE_ELEC.PY 15

IV. REPONSE_MECANIQUE.PY 18

V. REPONSE_MAGNETIQUE.PY 21

VI. DEFINITIONS DES CONSTANTES..... ERROR! BOOKMARK NOT DEFINED.

I. Modelisation.py

I.I. Fonction utiles

I.I.I. save_plot()

```python
def save_plot(fig_path: str, *plot_calls, title:str="", xlabel:str="", ylabel:str="", ylim=None,
xlim=None, legend:bool=True, log:bool=False, vspan:bool=False):
    """
    Enregistre une figure matplotlib avec des options de personnalisation.
    Args:
        fig_path (str): Chemin vers le fichier de sauvegarde
        plot_calls (tuple): Appels de tracé sous la forme (x, y, kwargs)
        title (str): Titre de la figure
        xlabel (str): nom de l'axe des x
        ylabel (str): nom de l'axe des y
        xlim (tuple): Limites de l'axe des x
        ylim (tuple): Limites de l'axe des y
        legend (bool): presence de la legende
        log (bool): presence d'une echelle logarithmique sur l'axe des x
        vspan (bool): presence des zones de frequences fondamentales et harmoniques
    """
    plt.figure(figsize=(6, 5))
    handles_curves, z_handles = [], []
    if vspan:
        z2 = plt.axvspan(82.41, 1174.66, color="#F0E68C", alpha=0.3,
                         label="Frequences fondamentales", zorder=0)
        z1 = plt.axvspan(1174.66, 5*1174.66, facecolor="none", alpha=0.5, hatch="////",
                         edgecolor="#424242", label="Frequences harmoniques", zorder=0)
        z_handles = [z1, z2]
    for call in plot_calls:
        x, y, kwargs = call
        line, = plt.plot(x, y, **kwargs)
        handles_curves.append(line)
    if title: plt.title(title)
    if xlabel: plt.xlabel(xlabel)
    if ylabel: plt.ylabel(ylabel)
    if ylim is not None: plt.ylim(ylim)
    if xlim is not None: plt.xlim(xlim)
    plt.grid(True, alpha=0.6) # Grid plus discrète pour le contraste
    if log: plt.xscale('log')
    if legend:
        leg_courbes = plt.legend(handles=handles_curves, loc='best', framealpha=1)
        if vspan:
            plt.gca().add_artist(leg_courbes)
            plt.legend(handles=z_handles, title="Zones d'etude", bbox_to_anchor=(0.5, -0.18),
                    loc='upper center', ncol=2, borderaxespad=0.)
    plt.tight_layout()
    plt.savefig(fig_path, dpi=300) # dpi=200 pour une nettete accrue en projection
    plt.close()
```

I.I.II. Normalize_signal()

```python
def normalize_signal(signal: np.ndarray):
    """
    Normalise un signal entre -1 et 1.
    Args :
        Signal (np.ndarray): Signal à normaliser
    Returns :
        Np.ndarray: Signal normaliser
    """
    max_val = np.max(np.abs(signal))
    if max_val == 0:
        return signal
    return signal / max_val
```

I.I.II. compute_and_normalize_fft()

```python
def compute_and_normalize_fft(signal: np.ndarray, Ne_fft: int):
    """
    Calcule et normalise l'amplitude du spectre de Fourier d'un signal.
    Args:
        signal (np.ndarray): Signal a analyser
        Ne_fft (int): Nombre de points dans la FFT
    """
    fft = np.fft.fft(signal)
    amp = np.abs(fft) / Ne_fft
    return amp
```

I.I.II. extraire _pics_separes()

```
def extraire_pics_separes(frequences, amplitudes, prominence=0.05, distance=10):
    """
    Extrait les pics significatifs d'un spectre de Fourier en utilisant la proeminence.
    Args:
        frequences (np.ndarray): Tableau des frequences
        amplitudes (np.ndarray): Tableau des amplitudes correspondantes
        prominence (float): Hauteur relative du pic par rapport à sa base locale.
                            Permet de detecter les petits pics aigus sans prendre le bruit fort.
        distance (int): Distance minimale entre deux pics pour les considérer comme séparés.
    Returns:
        np.ndarray: Liste des frequences
        np.ndarray: Liste des amplitudes
    """
    # Utilisation de 'prominence' au lieu de 'height' pour le problème d'amplitude variable
    indices, _ = find_peaks(amplitudes, prominence=prominence, distance=distance)
    # Extraction et conveRCCsion en listes
    liste_frequences = frequences[indices].tolist()
    liste_amplitudes = amplitudes[indices].tolist()
    return np.array(liste_frequences), np.round(liste_amplitudes, 3)
```

I.II. Algorithme de Levenberg-Marqardt

I.II.I. Z_micro()

```
def Z_micro(X:np.ndarray, Rs_S:float, w:np.ndarray)-> np.ndarray:
    """Calcul l'impedance d'un micro de guitare en une pulsation w
    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L
l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de Foucault
        RCCs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        w (np.ndarray): Tableau des pulsation w
    Returns:
        np.darray: Tableau Numpy contenant le module de l'impedance complexe |Z| en chaque pulsation
w
    """
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3
    # Parties reelle et imaginaire de l'admittance Y
    den_L = RCCs**2 + (L * w)**2
    Y_re = (1 / Rp) + (RCCs / den_L)
    Y_im = (C * w) - (L * w / den_L)
    return 1 / np.sqrt(Y_re**2 + Y_im**2)
```

I.II.II. Matrice_Jacobienne()

```
def Matrice_Jacobienne(X:np.ndarray, RCCs:float, f:np.ndarray)->np.ndarray:
    """Basee sur l'algorithme de Levenberg_Marquardt, cette fonction retourne la matrice Jacobienne
    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L
l'inductance [H], C la capacite parallèle [nF]et Rp la resistance parallèle [kOhm] modelisant les
courants de Foucault
        RCCs (float):Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
    Returns:
        np.ndarray: Matrice Jacobienne definie tel que chaque colonne i contient les derivees
partielles par rapport a la variable situe en i dans X pour toute
            pulsation w
    """
    w = 2 * np.pi * f
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3

    # Variables intermediaires
    den_L = RCCs**2 + (L * w)**2
    Y_re = (1 / Rp) + (RCCs / den_L)
    Y_im = (C * w) - (L * w / den_L)
    Z = 1 / np.sqrt(Y_re**2 + Y_im**2)
    Z3 = Z**3

    # Derivees partielles de Y
    dYre_dL = - (RCCs * 2 * L * w**2) / (den_L**2)
    dYim_dL = - (w * (RCCs**2 - (L * w)**2)) / (den_L**2)
    dZ_dL = - Z3 * (Y_re * dYre_dL + Y_im * dYim_dL)

    dYim_dCn = w * 1e-9
    dZ_dCn = - Z3 * (Y_im * dYim_dCn)

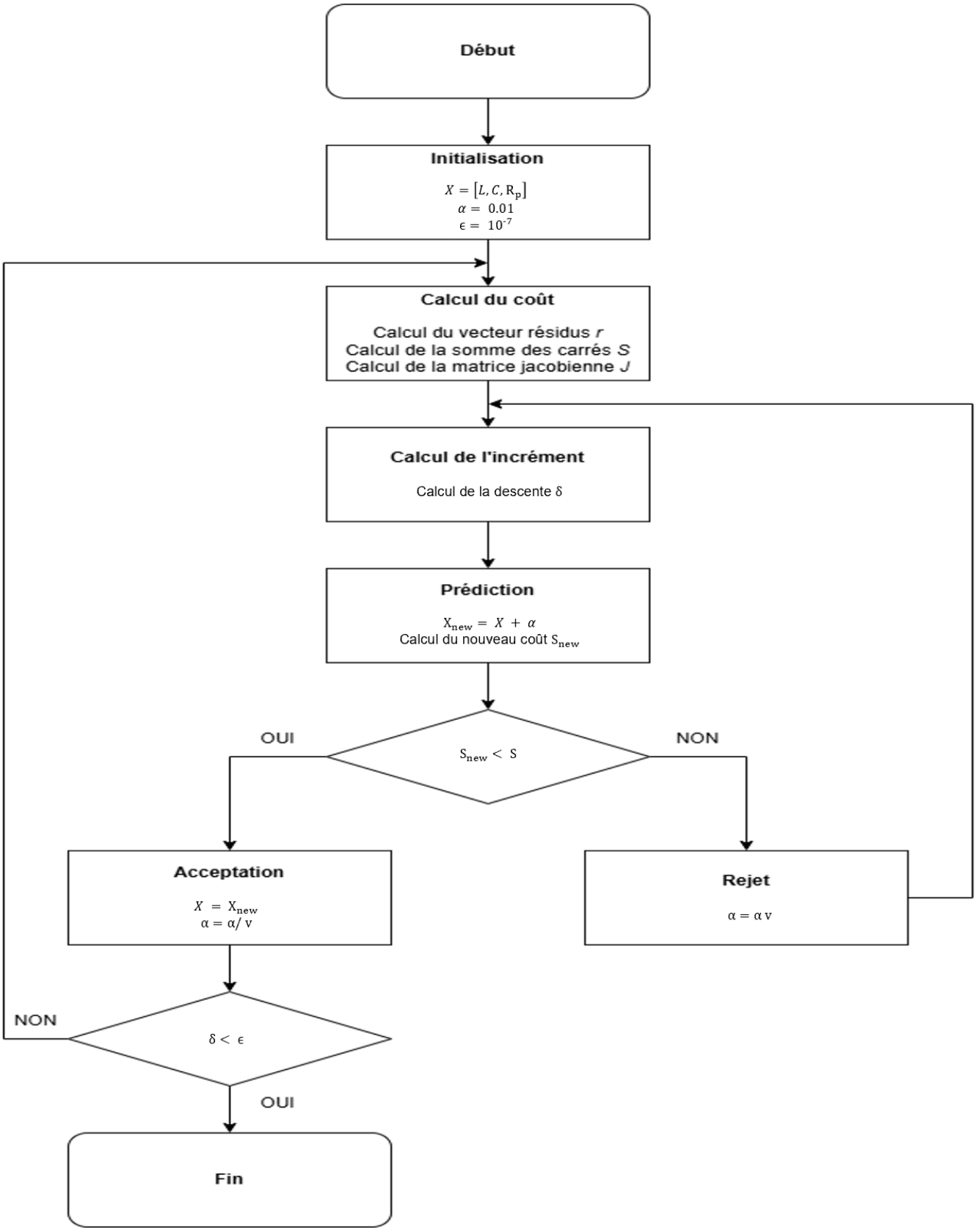
    dYre_dRpn = - 1 / (1000 * Rp_k**2)
    dZ_dRpk = - Z3 * (Y_re * dYre_dRpn)

    return np.column_stack((dZ_dL, dZ_dCn, dZ_dRpk))
```

I.II.III. residus()

```
def residus(X:np.ndarray, RCCs:float, f:np.ndarray, Z_exp:np.ndarray)-> np.ndarray:
    """Calcul l'ecart (l'erreur) entre l'impedance calcule theoriquement et l'impedance reelle
experimentale"""
    return Z_exp - Z_micro(X, RCCs, 2 * np.pi * f)
```

I.II.IV. Levenberg_marquardt()



```python
def Levenberg_Marquardt(X_init:np.ndarray, RCCs:float, f:np.ndarray, Z_exp:np.ndarray, N:int, learning_rate:float):
    """Algorithme de Levenberg-Marquardt pour l'optimisation non-lineaire des parametres electriques du micro
    Args:
        X_init (np.ndarray): Vecteur initial des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et Rp la resistance parallèle [kOhm] modelisant les courants de Foucault
        RCCs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Z_exp (np.ndarray): Tableau Numpy des impedances experimentale releve en Ohm
        N (int): Nombre d'iterations de l'algorithme
    Returns:
        np.ndarray: Vecteur optimisé des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de Foucault
    """
    X = np.array(X_init, dtype=float)
    alpha = learning_rate
    v = 3
    r = None
    eps = None
    for _ in range(N):
        r = residus(X, RCCs, f, Z_exp)
        eps = np.sum(r**2)
        J = Matrice_Jacobienne(X, RCCs, f)
        H = J.T @ J
        g = J.T @ r
        H_lm = H + alpha * np.diag(np.diag(H) + 1e-8)
        try:
            delta = np.linalg.solve(H_lm, g)
        except np.linalg.LinAlgError:
            break
        X_new = X + delta
        # Contrainte physique : paramètres toujouRCCs positifs
        X_new = np.maximum(X_new, [1e-3, 1e-3, 1e-3])
        eps_new = np.sum(residus(X_new, RCCs, f, Z_exp)**2)
        if eps_new < eps:
            alpha /= v
            X = X_new
            if np.linalg.norm(delta) / np.linalg.norm(X) < 1e-12:
                break
        else:
            alpha *= v
    return X, eps_new
```

I.II.V. calcul_bode()

```python
def calcul_bode(X:np.ndarray, RCCs:float, f:np.ndarray, Ramp:float=1e6)-> tuple[np.ndarray, np.ndarray]:
    """Calcul la fonction de transfert en gain et phase d'un micro de guitare en fonction de ses parametres internes
    Args:
        X (np.ndarray): Vecteur optimisé des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de Foucault
        RCCs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Ramp (float, optional): Resistance d'entree de l'amplificateur. Defaults to 1e6.
    Returns:
        tuple[np.ndarray, np.ndarray]: Deux tableaux Numpy contenant respectivement le gain (en dB) et la phase (en degres) de la fonction de transfert du micro
    """
    w = 2 * np.pi * f
    L, C, Rp = X[0], X[1] * 1e-9, X[2] * 1e3

    # Impedance serie
    Z_ser = RCCs + 1j * L * w

    # Impedance parallèle
    Y_para = (1 / Rp) + (1 / Ramp) + (1j * C * w)
    Z_para = 1 / Y_para

    # Fonction de transfert
    H = Z_para / (Z_ser + Z_para)

    gain = 20 * np.log10(np.abs(H))
    phase = np.angle(H, deg=True)
    return gain, phase
```

I.III. Fonctions pour la modélisation acoustique

I.III.I. fem_singlecoil()

```
def fem_singlecoil(n: int, t: np.ndarray, A0: float, x0: float) -> np.ndarray:
    """Force electromotrice induite dans le micro singlecoil (Théorie Robuste Non-Linéaire)
    Args:
        n (int): Nombre de modes consideres
        N (int): Nombre de spires de la bobine
        t (np.ndarray): Tableau des instants
        A0 (float): Amplitude initiale de la vibration
        x0 (float): Position de la perturbation
    Returns:
        np.ndarray: Signal de la f.e.m. induite (V)
    """

    # 1. Calcul des trajectoires vibratoires (Position et Vitesse)
    zs_t = np.full_like(t, z_0)
    vs_t = np.zeros_like(t)
    omega1 = 2 * np.pi * F

    for i in range(1, n + 1):
        Bn = (A0 * L**2.5 * np.sqrt(2)) / (i**2 * np.pi**2 * x0 * (L - x0)) * np.sin(i * np.pi * x0
/ L)

        omega_i = i * omega1
        spatial_node = np.sin(i * np.pi * x_s / L)

        zs_t += Bn * spatial_node * np.cos(omega_i * t)
        vs_t -= Bn * omega_i * spatial_node * np.sin(omega_i * t)

    # 2. Modélisation du champ magnétique et de son gradient
    B_ext = Bm_max * (Rm**3 / (Rm**2 + zs_t**2)**1.5)
    dB_dz = -3 * Bm_max * Rm**3 * zs_t / (Rm**2 + zs_t**2)**2.5

    # 3. Calcul du terme de transduction non-linéaire
    transduction_gain = (3 * B_ext / zs_t**4) - (1 / zs_t**3 * dB_dz)

    # 4. Calcul de la f.e.m. finale
    constante_flux = (N_s * S * chi * V) / (2 * np.pi)

    return constante_flux * transduction_gain * vs_t
```

I.III.II. fem_humbucker()

```
def fem_humbucker(n: int, N: int, x1: float, x2: float, t: np.ndarray, A0: float, x0: float) ->
np.ndarray:
    """Force electromotrice induite dans le micro Humbucker (Théorie Robuste Non-Linéaire)
    Args:
        n (int): Nombre de modes consideres
        N (int): Nombre de spires par bobine
        x1 (float): Position du 1er bobinage sur la corde
        x2 (float): Position du 2eme bobinage sur la corde
        t (np.ndarray): Tableau des instants
        A0 (float): Amplitude initiale de la vibration
        x0 (float): Position de la perturbation
    Returns:
        np.ndarray: Signal de la f.e.m. induite totale (V)
    """

    def calculate_coil_fem(x_pos):
        # 1. Calcul des trajectoires vibratoires (Position et Vitesse) à la position x_pos
        zs_t = np.full_like(t, z_0)
        vs_t = np.zeros_like(t)
        omega1 = 2 * np.pi * F

        for i in range(1, n + 1):
            Bn = (A0 * L**2.5 * np.sqrt(2)) / (i**2 * np.pi**2 * x0 * (L - x0)) * np.sin(i * np.pi *
x0 / L)

            omega_i = i * omega1
            spatial_node = np.sin(i * np.pi * x_pos / L)

            # Superposition des modes [cite: 2814, 2830]
            zs_t += Bn * spatial_node * np.cos(omega_i * t)
            vs_t -= Bn * omega_i * spatial_node * np.sin(omega_i * t)

        # 2. Modélisation du champ magnétique et de son gradient
        B_ext = Bm_max * (Rm**3 / (Rm**2 + zs_t**2)**1.5)
        dB_dz = -3 * Bm_max * Rm**3 * zs_t / (Rm**2 + zs_t**2)**2.5

        # 3. Calcul du terme de transduction non-linéaire G(z)
        transduction_gain = (3 * B_ext / zs_t**4) - (1 / zs_t**3 * dB_dz)

        # 4. Calcul de la f.e.m. élémentaire
        constante_flux = (N * S * chi * V) / (2 * np.pi)
        return constante_flux * transduction_gain * vs_t

    # Le Humbucker additionne les signaux utiles des deux bobines car
    # l'inversion de polarité magnétique compense l'inversion du bobinage
    return calculate_coil_fem(x1) + calculate_coil_fem(x2)
```

I.III.III. Atténuation & Filtrage spatial

```python
def attenuation_theorique_db(frequences: np.ndarray, x1: float, x2: float, xm: float) ->
tuple[np.ndarray, np.ndarray]:
    """Calcule la Fonction de Transfert Relative (Attenuation) du Humbucker
    par rapport au Single Coil en fonction de la frequence.
    Modèle : Filtrage en peigne dû à l'ecartement des bobines.
    Args:
        frequences (np.ndarray): Tableau des fréquences
        x1 (float): Position du 1er bobinage du micro humbucker sur la corde
        x2 (float): Position du 2ème bobinage du micro humbucker sur la corde
        xm (float): Position du micro singlecoil sur la corde
        L (float): Longueur de la corde
        F (float): Fréquence fondamentale de la corde
    Returns:
        tuple[np.ndarray, np.ndarray]: fonction de transfert en dB du Humbucker par rapport au
Single Coil, et atténuation due au filtre peigne seul
    """
    v = 2 * L * F
    k = 2 * np.pi * frequences / v
    delta_x = abs(x2-x1)
    x_humbucker = (x2+x1)/2

    signal_sc = np.abs(np.sin(k * xm))

    # Signal HB (Position + Filtre Peigne)
    # Note : x_humbucker est le centre du micro
    filtre_peigne = np.cos(k * delta_x / 2)
    signal_hb = np.abs(np.sin(k * x_humbucker)) * np.abs(filtre_peigne)

    # Ratio Theorique (Avec gestion de la division par zero)
    ratio = np.divide(signal_hb, signal_sc, out=np.zeros_like(signal_sc), where=signal_sc!=0)
    ratio_db = 20 * np.log10(ratio + 1e-6)
    return ratio_db, 20*np.log10(np.abs(np.cos(k*delta_x/2)))
```

I.IV. Fonction pour la modélisation magnétique

I.IV.I. fem_par_s()

```python
def fem_par_s(t: np.ndarray, B0: float, phi: float) -> np.ndarray:
     return N_s*S*2*np.pi*F_B*B0*np.sin(2*np.pi*F_B*t + phi)*mu_r
```

I.IV.II. fem_par_h()

```python
def fem_par_h(N: int, x1: float, x2: float, t: np.ndarray, B0: float, alpha: float) -> np.ndarray:
    return N*S*2*np.pi*F_B*B0*alpha*abs(x1-x2)*np.sin(2*np.pi*F_B*t)*mu_r
```

II. Traitement des données expérimentale

II.I. Filtrage des données

```python
import numpy as np
import matplotlib.pyplot as plt

def lire_fichier(PATH):
    List_temps = []
    List_sig = []
    file_s2 = open(PATH)
    L_lignes = file_s2.readlines()
    for ligne in L_lignes[2:]:
        ligne = ligne.strip().split(',')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace('E', 'e') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))
    return List_temps, List_sig

def write_file(PATH, List_temps, List_sig):
    with open(PATH, 'w') as file:
        file.write('Time;Signal\n')
        for t, s in zip(List_temps, List_sig):
            file.write(f"{t}\t{s}\n")
def filtre_passe_bas_1er_ordre(List_temps, List_sig, tau=0.001):
    """Filtre passe-bas du 1er ordre (IIR - Infinite Impulse Response).
    Params
    - List_temps: liste des temps
    - List_sig: liste des signaux
    - tau: constante de temps du filtre (en secondes).
    """
    sig_array = np.array(List_sig)

    # Calculer l'intervalle de temps moyen (en secondes)
    dt = (List_temps[-1] - List_temps[0]) / (len(List_temps) - 1)

    # Calculer le coefficient du filtre 1er ordre
    alpha = dt / (tau + dt)

    # Appliquer le filtre de manière causale : y[n] = alpha * x[n] + (1 - alpha) * y[n-1]
    filtered_sig = np.zeros_like(sig_array)
    filtered_sig[0] = sig_array[0]
    for i in range(1, len(sig_array)):
        filtered_sig[i] = alpha * sig_array[i] + (1 - alpha) * filtered_sig[i - 1]
    return filtered_sig

for i in range(len(L_path_in)):
    print(f"Fichier : {L_path_in[i]} : ...")
    L_temps, sig_brut = lire_fichier(L_path_in[i])
    write_file(L_path_out[i], L_temps, filtre_passe_bas_1er_ordre(sig_brut, L_temps))
    print(f"Fichier : {L_path_in[i]} : done")
```

II.II. Obtenir l'amplitude d'un signal périodique

```python
import numpy as np
import matplotlib.pyplot as plt

def write_file(PATH, List_temps, List_sig):
    with open(PATH, 'w') as file:
        file.write('Time;Signal\n')
        for t, s in zip(List_temps, List_sig):
            file.write(f"{t}\t{s}\n")

def get_amplitude(PATH):
    List_temps = []
    List_sig = []
    file = open(PATH)

    L_lignes = file.readlines()
    for ligne in L_lignes[1:70000]:
        ligne = ligne.strip().split('\t')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace(',', '.') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))

    amplitude = max(List_sig) - min(List_sig)
    return amplitude

for i in range(len(L_path)):
    amp = get_amplitude(L_path[i])
    write_file('Python/Code/amplitudes.csv', L_labels[:i+1], [get_amplitude(p) for p in
L_path[:i+1]])
```

III. Reponse_elec.py

```
# --- Importations des bibliothèques ---
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
from library import Modelisation as mod
import numpy as np
import matplotlib.pyplot as plt
import logging
plt.close("all")
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
# --- Variables ---
Rs_s = 5.26e3           # Resistance DC SingleCoil [Ohms]
Rs_hp = 4.52e3          # Resistance DC Humbucker parallèle [Ohms]
Rs_hs = 15.34e3         # Resistance DC Humbucker série [Ohms]
Ramp = 1e6              # Resistance de l'ampli [Ohms]
R = 97.6e3              # Resistance experimentale du PDT [Ohms]
X0 = np.array([2.5, 0.5, 450])    # Initalisation [L, C, R_p]
LR = 0.001              # Taux d'apprentissage
u_Ugbf = 0.01  # Incertitude sur tension GBF [V] (±1% lecture + résolution ADC)
u_res = 0.01   # Incertitude sur tension mesurée [V]
u_R = 97.6e3 * 0.01  # Incertitude sur R : ±1% (spec résistance de précision)
# --- Graphique ---
kw_Hp_th = {'color': '#000080', 'label': 'Humbucker parallèle - Théorie', 'linewidth': 2}
kw_S_th = {'color': '#8B0000', 'label': 'SingleCoil - Théorie', 'linewidth': 2}
kw_Hs_th = {'color': '#006400', 'label': 'Humbucker série - Théorie', 'linewidth': 2}
kw_Hp_exp = {'color': '#9400D3', 'label': 'Humbucker parallèle - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':'None'}
kw_Hs_exp = {'color': '#00CD00', 'label': 'Humbucker série - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':'None'}
kw_S_exp = {'color': '#FF8C00', 'label': 'SingleCoil - expérimentale', 'marker':"o", 'markersize':5,
'linestyle':'None'}

# --- Fonction pour lire les fichier experimentaux ---
def lire_fichier(PATH):
    f = []
    Vin = []
    Vs = []
    Vhp = []
    Vhs = []
    file = open(PATH)
    L_lignes = file.readlines()
    for ligne in L_lignes[1:]:
        ligne = ligne.strip().split(',')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace(',', '.') for x in ligne]
        Vin.append(float(ligne[0]))
        f.append(float(ligne[1]))
        Vs.append(float(ligne[2]))
        Vhp.append(float(ligne[3]))
        Vhs.append(float(ligne[4]))
    return np.array(Vin), np.array(f), np.array(Vs), np.array(Vhp), np.array(Vhs)
```

```python
# --- Fonction calculer les incertidude absolue ---
def calcul_incertitude_Z(R, u_R, U_gbf, u_Ugbf, u_res, u_ures):
    """
    Calcule l'incertitude ABSOLUE sur Z [Ohm].
    Formule : Z = R * (U_gbf / U_res - 1)
    Propagation d'erreur :
        u(Z)/Z = sqrt[(u_R/R)² + (u_Ugbf/U_gbf)² + (u_ures/U_res)²]
    """
    # Incertitude relative
    u_Z_rel = np.sqrt((u_R / R)**2 +
                      (u_Ugbf / U_gbf)**2 +
                      (u_ures / u_res)**2)
    # Impédance pour chaque point
    Z_calc = R * (U_gbf / u_res - 1)
    # Incertitude absolue [Ohm]
    u_Z_abs = Z_calc * u_Z_rel
    return u_Z_abs

# --- Lectures des données experimentales ---
f, Vin, Vs, Vhp, Vhs = lire_fichier("Python/data/raw/01-Reponse_elec/Mesure_Impedance_PDT_100k.csv")
Z_s = R * (21.1/Vs - 1)
Z_hp = R * (21.1/Vhp - 1)
Z_hs = R * (21.1/Vhs - 1)

u_Z_s = calcul_incertitude_Z(R, u_R, 21.1, u_Ugbf, Vs, u_res) * 1e-3  # kOhm
u_Z_hp = calcul_incertitude_Z(R, u_R, 21.1, u_Ugbf, Vhp, u_res) * 1e-3  # kOhm
u_Z_hs = calcul_incertitude_Z(R, u_R, 21.1, u_Ugbf, Vhs, u_res) * 1e-3  # kOhm
kw_S_exp['yerr'] = u_Z_s
kw_Hp_exp['yerr'] = u_Z_hp
kw_Hs_exp['yerr'] = u_Z_hs

# --- Affichage des incertitudes ---
print("\n" + "="*60)
print("INCERTITUDES SUR L'IMPEDANCE [kOhm]")
print(f"SingleCoil:")
print(f"  - Incertitude relative: {np.mean(u_Z_s / (Z_s*1e-3))*100:.2f}%")
print(f"\nHumbucker parallèle:")
print(f"  - Incertitude relative: {np.mean(u_Z_hp / (Z_hp*1e-3))*100:.2f}%")
print(f"\nHumbucker série:")
print(f"  - Incertitude relative: {np.mean(u_Z_hs / (Z_hs*1e-3))*100:.2f}%")

# --- Appel de l'algorithme de Levenberg-Marquardt ---
L_S = np.zeros((3,30))
for i in range(1,31):
    X_s, S_s = mod.Levenberg_Marquardt(X0, Rs_s, f, Z_s, i, LR)
    X_hp, S_hp = mod.Levenberg_Marquardt(X0, Rs_hp, f, Z_hp, i, LR)
    X_hs, S_hs = mod.Levenberg_Marquardt(X0, Rs_hs, f, Z_hs, i, LR)
    L_S[0, i-1] = S_s
    L_S[1, i-1] = S_hp
    L_S[2, i-1] = S_hs
L_S[0] = mod.normalize_signal(L_S[0])
L_S[1] = mod.normalize_signal(L_S[1])
L_S[2] = mod.normalize_signal(L_S[2])
```

```
print(f"SingleCoil: L={X_s[0]:.2f}H, C={X_s[1]:.2f}nF, Rp={X_s[2]:.1f}kOhm")
print(f"Humbucker parallele: L={X_hp[0]:.2f}H, C={X_hp[1]:.2f}nF, Rp={X_hp[2]:.1f}kOhm")
print(f"Humbucker parallele: L={X_hs[0]:.2f}H, C={X_hs[1]:.2f}nF, Rp={X_hs[2]:.1f}kOhm")

# --- Modèle numérique ---
freq = np.linspace(10, 30_000, 1000)
gain_s, phase_s = mod.calcul_bode(X_s, Rs_s, freq, Ramp)
gain_hp, phase_hp = mod.calcul_bode(X_hp, Rs_hp, freq, Ramp)
gain_hs, phase_hs = mod.calcul_bode(X_hs, Rs_hs, freq, Ramp)

# --- Diagramme de Bode ---
plt.figure(figsize=(8,6))
plt.subplot(2, 1, 1)
plt.axvspan(82.41, 1174.66, color="#F0E68C", alpha=0.3, label="Frequences fondamentales", zorder=0)
plt.axvspan(1174.66, 5*1174.66, facecolor="none", alpha=0.5, hatch="////",
                        edgecolor="#424242", label="Frequences harmoniques", zorder=0)
plt.plot(freq, gain_s, **kw_S_th)
plt.plot(freq, gain_hp, **kw_Hp_th)
plt.plot(freq, gain_hs, **kw_Hs_th)
plt.xscale('log')
plt.grid(which='both', linestyle='--', linewidth=0.5)
plt.ylabel("Gain [$dB$]")
plt.title("Diagramme de Bode des micros")
plt.xlabel("Frequence [$Hz$]")
plt.xlim((10, 30000))
plt.subplot(2, 1, 2)
plt.axvspan(82.41, 1174.66, color="#F0E68C", alpha=0.3, label="Frequences fondamentales", zorder=0)
plt.axvspan(1174.66, 5*1174.66, facecolor="none", alpha=0.5, hatch="////",
                        edgecolor="#424242", label="Frequences harmoniques", zorder=0)
plt.plot(freq, phase_s, **kw_S_th)
plt.plot(freq, phase_hp, **kw_Hp_th)
plt.plot(freq, phase_hs, **kw_Hs_th)
plt.xscale('log')
plt.grid(which='both', linestyle='--', linewidth=0.5)
plt.ylabel("Phase [$°$]")
plt.xlabel("Frequence [$Hz$]")
plt.xlim((10, 30000))
plt.legend(loc='lower left')
plt.savefig("Python/Exports/01-Reponse_electrique_frequetielle/00-Bode.png")
plt.close()

# --- Courbe Z = f(w) ---
mod.save_plot("Python/Exports/01-Reponse_electrique_frequetielle/Impedance.png",
                (f, Z_s*1e-3, kw_S_exp),
                (f, Z_hp*1e-3, kw_Hp_exp),
                (f, Z_hs*1e-3, kw_Hs_exp),
                (freq, mod.Z_micro(X_s, Rs_s, 2*np.pi*freq)*1e-3, kw_S_th),
                (freq, mod.Z_micro(X_hp, Rs_hp, 2*np.pi*freq)*1e-3, kw_Hp_th),
                (freq, mod.Z_micro(X_hs, Rs_hs, 2*np.pi*freq)*1e-3, kw_Hs_th),
                title=r"Impedances des micros $|Z|$ en fonction de la frequence",
                xlabel="Frequence [$Hz$]", ylabel=r"Impedance [$k\Omega$]", vspan=True,
                xlim=(10, 30_000), legend=True, error_bars=True)
```

IV. Reponse_mecanique.py

```python
# --- Importations des bibliothèques ---
import os
import sys
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
from library import Modelisation as mod
import numpy as np
import matplotlib.pyplot as plt
import logging
plt.close("all")
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
# --- Constante pour code ---
tracer_fft = True
PATH_exp = "Python/Exports/02-Reponse_mecanique/"
Na = 20         # Nombre de mode considérés
Te = 1          # Temps d'echentillonage [s]
Ne = 10000      # Nombre de point d'echantillonage
fs = Ne / Te    # Frequence d'echantillonage [Hz]
yerr_temp = (1e-5 / 2 )         # Incertitude de mesure oscillo [V]
# --- Variables ---
mu_0 = 4e-7 * np.pi         # Permeabilite magnetique du vide [H.m-1]
mu_r = 3.5                  # Permeabilite relative de l'Alnico 5 [.]
K=0.7                       # Coeffient K [.]
x_hp1, x_hp2 = 0.147, 0.165 # Position du humbucker parallèle [m]
x_hs1, x_hs2 = 0.202, 0.220 # Position du humbucker série [m]
S = 1.5e-4                  # Surface d'une bobine [m2]
Rs_s = 5.26e3               # Resistance DC du SingleCoil [Ohm]
L_s = 2.43                   # Inductance du SingleCoil [H]
N_s = np.sqrt(L_s * 1.5e-2 / (mu_0 * mu_r * S * K))
L_hp = 1.77                 # Inductance du Humbucker parallèle [H]
N_hp = np.sqrt(2*L_hp * 1.5e-2 / (mu_0 * mu_r * S * K))
L_hs = 4.71                 # Inductance du Humbucker parallèle [H]
N_hs = np.sqrt(2*L_hs * 1.5e-2 / (mu_0 * mu_r * S * K))
A0 = 1e-3                   # Amplitude initiale de vibration [m]
x0 = 0.300                  # Origine de la perturbation [m]
F = 80                      # Frequence fondamentale [Hz]

# --- Couleurs graphiques ---
kw_Hp_th = {'color': '#000080', 'label': 'Humbucker parallèle - Théorie', 'linewidth': 2}
kw_S_th = {'color': '#8B0000', 'label': 'SingleCoil - Théorie', 'linewidth': 2}
kw_Hs_th = {'color': '#006400', 'label': 'Humbucker série - Théorie', 'linewidth': 2}
kw_Hp_exp = {'color': '#9400D3', 'label': 'Humbucker parallèle - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':':'}
kw_Hs_exp = {'color': "#00AB00", 'label': 'Humbucker série - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':':'}
kw_S_exp = {'color': '#FF8C00', 'label': 'SingleCoil - expérimentale', 'marker':"o", 'markersize':5,
'linestyle':':'}
kw_peigne = {'color': '#006400', 'label': 'Peigne Spatial  Théorie', 'linewidth': 2}
kw_peigne_exp = {'color': "#00AB00", 'label': 'Peigne spatial - Experience', 'marker':"o",
'markersize':5, 'linestyle':':'}
Title = "Spectre de la tension en sortie des micros\n(Fondamental à " + str(F) + " Hz)"
```

```python
# --- Lecture des données experimentales ---
def lire_fichier(PATH):
    List_temps = []
    List_sig = []
    file_s2 = open(PATH)
    L_lignes = file_s2.readlines()
    for ligne in L_lignes[2:]:
        ligne = ligne.strip().split(',')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace('E', 'e') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))
    return np.array(List_temps), np.array(List_sig)
temps_s, V_s_exp = lire_fichier("Python/data/raw/02-Reponse_meca/S2.csv")
temps_hp, V_hp_exp = lire_fichier("Python/data/raw/02-Reponse_meca/Hp2.csv")
temps_hs, V_hs_exp = lire_fichier("Python/data/raw/02-Reponse_meca/Hs3.csv")

# --- Calcul des FFT ---
temps = np.linspace(-Te/2, Te/2, Ne)
V_s_th = mod.fem_singlecoil(Na, temps, A0, x0)
V_hp_th = mod.fem_humbucker(Na, N_hp, x_hp1, x_hp2, temps, A0, x0) / 2
V_hs_th = mod.fem_humbucker(Na, N_hs, x_hs1, x_hs2, temps, A0, x0) / 2

fft_s_th = mod.compute_and_normalize_fft(V_s_th, Ne)
fft_hp_th = mod.compute_and_normalize_fft(V_hp_th, Ne)
fft_hs_th = mod.compute_and_normalize_fft(V_hs_th, Ne)

# Nombre d'echantillon experimental
Ne_s = len(V_s_exp)
Ne_hp = len(V_hp_exp)
Ne_hs = len(V_hs_exp)

fft_s_exp_abs = mod.compute_and_normalize_fft(V_s_exp, Ne_s)
fft_hp_exp_abs = mod.compute_and_normalize_fft(V_hp_exp, Ne_hp)
fft_hs_exp_abs = mod.compute_and_normalize_fft(V_hs_exp, Ne_hs)

# Propager l'incertitude CSV vers la FFT (division par √N)
yerr_s_fft_abs = yerr_temp / np.sqrt(Ne_s)
yerr_hp_fft_abs = yerr_temp / np.sqrt(Ne_hp)
yerr_hs_fft_abs = yerr_temp / np.sqrt(Ne_hs)

# Tableau de fréquence pour chaque données
freqs_th = np.fft.fftfreq(Ne, 1 / (len(V_s_th) / (temps[-1] - temps[0])))
freqs_s_exp = np.fft.fftfreq(Ne_s, 1 / (len(V_s_exp) / (temps_s[-1] - temps_s[0])))
freqs_hp_exp = np.fft.fftfreq(Ne_hp, 1 / (len(V_hp_exp) / (temps_hp[-1] - temps_hp[0])))
freqs_hs_exp = np.fft.fftfreq(Ne_hs, 1 / (len(V_hs_exp) / (temps_hs[-1] - temps_hs[0])))

# Masque d'affichage entre 40Hz et 2000Hz
mask_th = (40 < freqs_th) & (freqs_th <= 2000)
mask_s = (40 < freqs_s_exp) & (freqs_s_exp <= 2000)
mask_hp = (40 < freqs_hp_exp) & (freqs_hp_exp <= 2000)
mask_hs = (40 < freqs_hs_exp) & (freqs_hs_exp <= 2000)
freqs_th = freqs_th[mask_th]
fft_s_th = mod.normalize_signal(fft_s_th[mask_th])
```

```
fft_hp_th = mod.normalize_signal(fft_hp_th[mask_th])
fft_hs_th = mod.normalize_signal(fft_hs_th[mask_th])

# Normaliser les FFT ET l'incertitude
max_fft_s = np.max(fft_s_exp_abs)
max_fft_hp = np.max(fft_hp_exp_abs)
max_fft_hs = np.max(fft_hs_exp_abs)

freqs_s_exp = freqs_s_exp[mask_s]
fft_s_exp_filtered = fft_s_exp_abs[mask_s]
fft_s_exp = fft_s_exp_filtered / max_fft_s
yerr_s_exp = np.full_like(fft_s_exp_filtered, yerr_s_fft_abs) / max_fft_s

freqs_hp_exp = freqs_hp_exp[mask_hp]
fft_hp_exp_filtered = fft_hp_exp_abs[mask_hp]
fft_hp_exp = fft_hp_exp_filtered / max_fft_hp
yerr_hp_exp = np.full_like(fft_hp_exp_filtered, yerr_hp_fft_abs) / max_fft_hp

freqs_hs_exp = freqs_hs_exp[mask_hs]
fft_hs_exp_filtered = fft_hs_exp_abs[mask_hs]
fft_hs_exp = fft_hs_exp_filtered / max_fft_hs
yerr_hs_exp = np.full_like(fft_hs_exp_filtered, yerr_hs_fft_abs) / max_fft_hs

# --- Affichage des incertitudes moyennes sur les FFT ---
print("INCERTITUDES SUR LES FFT (DONNÉES NORMALISÉES)")
print(f"SingleCoil:")
print(f"  - Incertitude relative: {np.mean(yerr_s_exp / (fft_s_exp + 1e-12))*100:.4f}%")
print(f"\nHumbucker parallèle:")
print(f"  - Incertitude relative: {np.mean(yerr_hp_exp / (fft_hp_exp + 1e-12))*100:.4f}%")
print(f"\nHumbucker série:")
print(f"  - Incertitude relative: {np.mean(yerr_hs_exp / (fft_hs_exp + 1e-12))*100:.4f}%")

# --- Enregistrer les spectres ---
mod.save_plot(PATH_exp + "FFT/Experience.png",
        (freqs_s_exp, fft_s_exp, {**kw_S_exp, 'yerr': yerr_s_exp}),
        (freqs_hp_exp, fft_hp_exp, {**kw_Hp_exp, 'yerr': yerr_hp_exp}),
        (freqs_hs_exp, fft_hs_exp, {**kw_Hs_exp, 'yerr': yerr_hs_exp}),
        title=Title, xlabel="Frequence [$Hz$]", ylabel="Tension normalisee", error_bars=True)
print(f"FFT sauvegardees sous {PATH_exp + "FFT/Experience.png"}")

# --- Mise en evidence du filtrage spatial ---
freqs = np.linspace(10, 20000, 1000)
L_freqs, L_pic_s_exp = mod.extraire_pics_separes(freqs_s_exp, fft_s_exp, prominence=0.04,
distance=5)
L_freqs, L_pic_h_exp = mod.extraire_pics_separes(freqs_hs_exp, fft_hs_exp, prominence=0.03,
distance=5)
peigne_exp = 20*np.log10(L_pic_h_exp / L_pic_s_exp)
ratio_th_hs, peigne_hs = mod.attenuation_theorique_db(freqs, x_hs1, x_hs2)

mod.save_plot(PATH_exp + "Filtrage_spatial.png",
    (freqs, ratio_th_hs, {'color': "#444444", 'label': "Gain Theorique S [$dB$]", "linewidth":
0.5}), (L_freqs, peigne_exp, kw_peigne_exp), (freqs, peigne_hs, kw_peigne))
```

V. Reponse_magnetique.py

```python
# --- Importations des bibliothèques ---
import os
import sys
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
from library import Modelisation as mod
import numpy as np
import matplotlib.pyplot as plt
import logging
plt.close("all")
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')


# --- Constantes code ---
PATH_out = "Python/Exports/03-Reponse_magnetique/"
Te = 4000 * 0.00001     # Temps d'echantillonage [s]
Ne = 4000               # Nombre d'echantillon [.]

# --- Variable ---
mu_0 = 4e-7 * np.pi         # Permeabilite magnetique du vide [H.m-1]
mu_r = 3.5                  # Permeabilite relative de l'Alnico 5 [.]
K = 0.7                     # Coeffient K [.]
x_hp1, x_hp2 = 0.147, 0.165 # Position du micro humbucker [m]
S = 1.5e-4                  # Surface de la bobine du micro [m2]
Rs_s = 5.26e3               # Resistance DC du SingleCoil [Ohm]
L_s = 2.43                   # Inductance du SingleCoil [H]
N_s = np.sqrt((L_s * 1.5e-2) / (mu_0 * mu_r * S * K))
L_hp = 1.77                  # Inductance du micro [H]
N_hp = np.sqrt(L_hp * 1.5e-2 / (mu_0 * mu_r * S * K)) / 2
L_hs = 4.71                  # Inductance du micro [H]
N_hs = np.sqrt(L_hs * 1.5e-2 / (mu_0 * mu_r * S * K)) / 2
FB = 50                     # Frequence du champ parasite [Hz]
B0 = 46 * 10**(-6)          # Champ magnetique au centre des bobines Helmholtz [T]
alpha = 4                   # Coefficient alpha [m-1]
Delta = 0.018

# --- Couleurs graphiques ---
kw_Hp_th = {'color': '#000080', 'label': 'Humbucker parallèle - Théorie', 'linewidth': 2}
kw_S_th = {'color': '#8B0000', 'label': 'SingleCoil - Théorie', 'linewidth': 2}
kw_Hs_th = {'color': '#006400', 'label': 'Humbucker série - Théorie', 'linewidth': 2}
kw_Hp_exp = {'color': '#9400D3', 'label': 'Humbucker parallèle - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':'None'}
kw_Hs_exp = {'color': '#00CD00', 'label': 'Humbucker série - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':'None'}
kw_S_exp = {'color': '#FF8C00', 'label': 'SingleCoil - expérimentale', 'marker':"o",
'markersize':5, 'linestyle':'None'}

# --- Données experimentales ---
List_Bpar = np.array([0, 5.5, 11, 16.5, 22, 27.5, 33, 38.5, 44, 49.5]) * 10**-6
```

```python
def _read_amplitude_file(path): # Lire les fichiers contenant les données
    s_list = [0]
    hp_list = [0]
    with open(path, 'r') as fh:
        lines = fh.readlines()
    for ligne in lines[1:]:
        cols = ligne.strip().split('\t')
        cols = [c.replace('\n', '') for c in cols]
        cols = [c.replace(',', '.') for c in cols]
        label, amp_s = cols[0], float(cols[1])
        if "Humbucker" in label:
            hp_list.append(amp_s)
        elif "Single Coil" in label:
            s_list.append(amp_s)
    return np.array(hp_list), np.array(s_list)

List_amp_hp_exp, List_amp_s_exp =
_read_amplitude_file("Python/data/filter/!Reponse_magnetique_amplitude.csv")

# ajustement linéaire expérimental
k_h_exp, _ = np.polyfit(List_Bpar, List_amp_hp_exp, 1)
k_s_exp, _ = np.polyfit(List_Bpar, List_amp_s_exp, 1)
List_amp_hp_exp_lin = k_h_exp * List_Bpar
List_amp_s_exp_lin = k_s_exp * List_Bpar


# --- Données Théorique ---
ks = N_s * S * 2 * np.pi * FB * mu_r
khp = 0.5 * N_hp * S * 2 * np.pi * FB * mu_r * alpha * Delta
khs = N_hs * S * 2 * np.pi * FB * mu_r * alpha * Delta

List_amp_s_th = ks * List_Bpar
List_amp_hp_th = khp * List_Bpar
List_amp_hs_th = khs * List_Bpar

# --- Parrallèle ---
CMRRp_exp = 20 * np.log10(k_s_exp / k_h_exp)
CMRRp_th = 20 * np.log10(ks / khp)
tauxp_th = ks / khp
tauxp_exp = k_s_exp / k_h_exp

# --- Série ---
#CMRRp_exp = 20*np.log10(k_s_exp / k_hs_exp)
CMRRs_th = 20 * np.log10(ks / khs)
tauxs_th = ks / khs
#tauxp_exp = k_s_exp / k_hs_exp

# --- Ecarts ---
eps_cmrrp = abs((CMRRp_th - CMRRp_exp) / (CMRRp_th)) * 100
#eps_cmrrs = abs((CMRRs_th - CMRRs_exp)/(CMRRs_th)) *100

# --- Droite sensibilité ---
mod.save_plot(PATH_out + "Droite_de_sensibilite.png",
        (List_Bpar*1e6, List_amp_s_th*1e3, kw_S_th),
```

```
        (List_Bpar*1e6, List_amp_s_exp*1e3, kw_S_exp),
        (List_Bpar*1e6, List_amp_hp_th*1e3, kw_Hp_th),
        (List_Bpar*1e6, List_amp_hp_exp*1e3, kw_Hp_exp),
        (List_Bpar*1e6, List_amp_hs_th*1e3, kw_Hs_th),
        title="Tension induite dans le micro en fonction d'un \nchamp magnetique exterieur à 50
$Hz$",
        xlabel="Amplitude du champ parasite [$µT$]", ylabel="Amplitude Tension [$mV$]")

print(f"k_s = {ks*10**-3} mV/microTesla")
print(f"k_hp = {khp*10**-3} mV/microTesla")
print(f"k_hs = {khs*10**-3} mV/microTesla")

print(f"Taux theorique P = {tauxp_th}")
print(f"CMRR theorique P = {CMRRp_th} dB")
print(f"Taux theorique S = {tauxs_th}")
print(f"CMRR theorique S = {CMRRs_th} dB")


print("\nCalcul des sensibilites et CMRR experimental")
print(f"k_s_exp = {k_s_exp*10**-3} mV/microTesla")
print(f"k_h_exp = {k_h_exp*10**-3} mV/microTesla")
print(f"Taux exp = {tauxp_exp}")
print(f"CMRR_exp = {20*np.log10( k_s_exp / k_h_exp )} dB")
print("\nCalcul des erreurs relatives...")

print(f"Erreur relative cmrrp : {eps_cmrrp} %")
#print(f"Erreur relative cmrrs : {eps_cmrrs} %")
```